

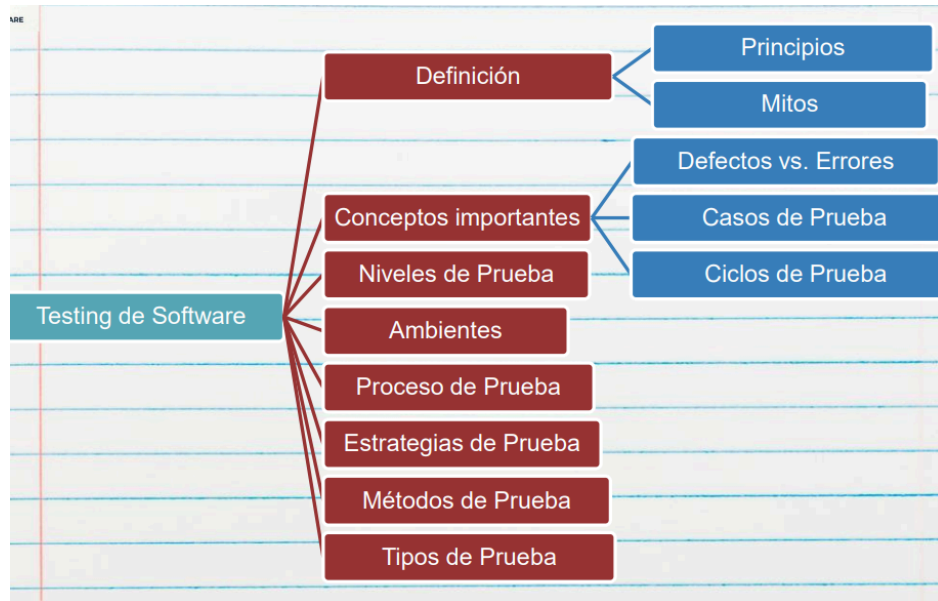
parcial 2

Testing de software / Prueba de software

El desafío del testing es encontrar defectos. No asegura ni garantiza calidad. No garantiza que el producto no tenga defectos, los hace visibles pero no asegura que no hayan más.

La percepción de calidad es una cuestión subjetiva. Se define como el nivel de cumplimiento a las necesidades y expectativas → requerimientos.

El testing se realiza cuando se tiene parte del producto construida.



Es un proceso destructivo que trata de encontrar defectos en el software, su objetivo no es asegurar que el software funcione, sino encontrar defectos, para lo que asumimos que en el software HAY defectos.

No debemos pensar que no hay defectos ni que debemos verificar que no los hay, sino que empezamos con la hipótesis o la actitud negativa de que el software es incorrecto.

Decimos entonces, que el test va a ser exitoso cuando encontremos defectos

Definición de prueba de software

- Proceso destructivo de tratar de encontrar defectos, cuya presencia se asume, en el código.
- Se debe ir con una actitud negativa para demostrar que algo es incorrecto.
- Un testing exitoso es el que encuentra defectos.

Asegurar calidad vs Controlar la calidad

Una vez definidos los requerimientos de calidad, tengo que tener en cuenta que:

- La calidad no puede "inyectarse" al final
- La calidad del producto depende de tareas realizadas durante todo el proceso

- Detectar errores en forma temprana ahorra esfuerzos, tiempo, recursos
- La calidad no solamente abarca aspectos del producto sino también del proceso y como éstos se pueden mejorar, que a su vez evita defectos recurrentes.
- El testing NO puede asegurar ni calidad en el software ni software de calidad

Defectos vs Errores vs Falla

- El **error** se descubre a partir de técnicas específicas que me permiten encontrar los mismos dentro de la misma etapa en la que estoy trabajando. Siempre es mejor encontrar errores antes que defectos.
- Los **defectos** nos demuestran un error no detectado que se trasladó a una etapa siguiente. En el testing encontramos defectos, ya que justamente se encuentran cosas incorrectas que se realizaron en la etapa de implementación que es la etapa anterior.
- La **falla** es un mal funcionamiento del sistema, ocurre como consecuencia de un error o defecto
- No todos los defectos provocan fallas en el software → defectos que no provocan un mal funcionamiento de sistema, por ejemplo → defectos de diseño de interfaz, errores de ortografía

Severidad

Tiene que ver con la gravedad del defecto.

1. Bloqueante o invalidante → no se puede usar el sistema.
2. Crítico → no puedo cumplir mi objetivo con cierta funcionalidad (y no hay un plan B), pero no me quedo sin sistema.
3. Mayor → no puedo cumplir mi objetivo con cierta funcionalidad pero tengo forma de solucionarlo
4. Menor → no te impide el cumplimiento de la funcionalidad pero hay errores de baja importancia
5. Cosmético → errores de ortografía, formatos de fechas, distribución de componentes

Prioridad

Urgencia que tenemos para resolver este defecto.
La define el usuario final (si es que lo detecta)

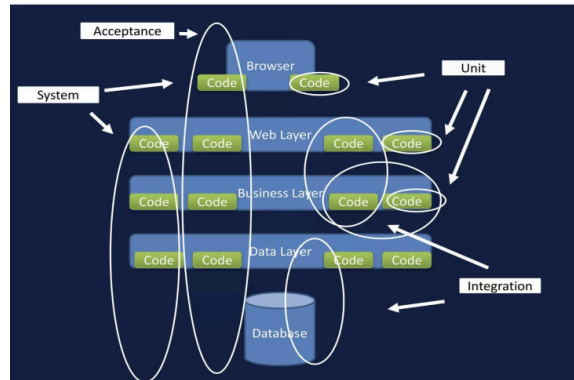
1. Urgencia
2. Alta
3. Media
4. Baja

Estas características son independientes entre sí, ya que dependen del tipo de negocio y de sistema que se está desarrollando. Por ejemplo, un defecto cosmético puede tener baja prioridad para un sistema de inventarios, pero puede tener una alta prioridad si se trata de un sistema de una empresa de marketing, en donde el aspecto y la presentación es algo muy importante.

Niveles de prueba

1. Pruebas unitarias → realizadas por el desarrollador. Son aquellas en donde pruebo un componente individual, algo acotado que tiene que ver con el desarrollo que estoy realizando. Son respecto a aspectos puntuales y aislados del proyecto que se está realizando. Al probar componentes individuales y de forma independiente, normalmente se ejecutan por el mismo desarrollador. En estas pruebas se encuentran errores más que defectos. Se encuentran en el límite. Es muy fácil de automatizar. Se realizan en el workflow de implementación.
2. Pruebas de integración o de interfaces → implica integrar los componentes ya probados en las pruebas unitarias. Se integran para ver su funcionamiento conjunto. Normalmente requieren de un tester. Como estamos buscando verificar que las partes aisladas funcionan bien en conjunto, se hace una integración de manera incremental para lograr una identificación más correcta de los errores. Se realizan en el workflow de implementación.

3. Pruebas de Sistema → son más amplias, no solo pruebo la integración entre dos componentes, sino el sistema en toda su escala. Busca asegurarse que el sistema en su totalidad funcione de manera satisfactoria. Empiezan a impactar otras cuestiones que tienen que ver no solo con requerimientos funcionales, sino también con requerimientos no funcionales. El ambiente de prueba tiene que ser lo más parecido al entorno de producción. Se realizan en el workflow de prueba.
4. Pruebas de aceptación de usuario → se realizan en el workflow de despliegue. Son ejecutadas por el usuario final o el cliente. Busca que el cliente/usuario se familiarice con el producto, generando comodidad y confianza sobre el mismo (su objetivo principal no es encontrar fallas). En estas pruebas, el ambiente también tiene que ser lo más parecido posible al entorno de producción.



Ambientes (Entornos) para la construcción del software

Es la configuración del espacio de trabajo y todas las herramientas o recursos que se necesitan para el desarrollo. Cada ambiente cumple un propósito específico y se utiliza en diferentes etapas del proceso de desarrollo y despliegue

- Desarrollo → el ambiente de desarrollo es donde los desarrolladores crean, prueban y depuran el software. Es un entorno local o en red utilizado por los desarrolladores para escribir y probar el código antes de integrarlo con el resto del sistema. Los programadores pueden utilizar herramientas de desarrollo, depuración y pruebas para garantizar que el software cumpla con los requisitos establecidos. Este ambiente suele ser flexible y permite a los desarrolladores experimentar y probar nuevas ideas sin afectar los entornos de producción. Las pruebas unitarias se llevan a cabo en el ambiente de desarrollo
- Prueba → el ambiente de pruebas es donde se llevan a cabo pruebas exhaustivas del software desarrollado. Aquí se realizan pruebas de integración, pruebas funcionales, pruebas de rendimiento y otras pruebas relevantes para verificar que el software cumpla con los requisitos establecidos y funcione correctamente. El ambiente de prueba suele ser una réplica o tener características parecidas al entorno de producción, pero sin afectar a los usuarios finales y sin TODAS las características (ya que es caro tener ambientes iguales al de producción). Se utilizan conjuntos de datos y configuraciones similares a las del entorno de producción para garantizar una prueba más precisa del software. Las pruebas de integración se realizan en el ambiente de pruebas
- Pre-Producción → el ambiente de preproducción, también conocido como entorno de puesta en escena o entorno de calidad, es donde se realiza una prueba final del software antes de su lanzamiento en producción. Aquí se simulan las condiciones del entorno de producción y se realizan pruebas de último minuto, como pruebas de estrés, pruebas de carga y pruebas de seguridad. El objetivo es validar y verificar que el software esté listo para ser implementado en el entorno de producción sin problemas significativos. Las pruebas de sistema se llevan a cabo en el ambiente de preproducción

- Producción → el ambiente de producción es donde el software está en funcionamiento y es accesible a los usuarios finales. Es el entorno real en el que el software se utiliza para realizar las tareas y funciones para las que fue diseñado. Este ambiente suele ser altamente controlado y está configurado para ser escalable, seguro y confiable. Se implementan medidas de respaldo y recuperación ante desastres para garantizar la disponibilidad continua del software. Es el ambiente en el que el software está funcionando. Las pruebas de aceptación se llevan a cabo en el ambiente de preproducción o en un entorno similar al de producción (en el ambiente de prueba)

Caso de prueba

- Set de condiciones o variables bajo las cuales un tester determinará si el software está funcionando correctamente o no. Tiene que ver con ejecutar una serie de pasos o acciones en una determinada funcionalidad determinando completamente con que valores voy a hacer la ejecución para ver si el software me va a dar los resultados esperados o no. Además de especificar cuál es la acción a ejecutar, me debe especificar con que datos debo realizarlo.
- Buena definición de casos de prueba nos ayuda a REPRODUCIR defectos
- Testear sin casos de pruebas es una pérdida de tiempo
- Un problema es que el testing es caro y pruebas exhaustivas no se pueden hacer (probar todas las combinaciones). Debo tener algún mecanismo para lograr definir la menor cantidad de casos de prueba, pero que estos me permitan cubrir el testing completamente, o de la manera más completa posible.
- Se busca reducir la cantidad de horas operativas

Etapas	Planificación de pruebas	Diseño de casos de pruebas	Ejecución	Reporte de defectos
Artefacto	Plan de pruebas	Casos de pruebas Datos de pruebas	Version de software	Reporte Defectos